

```
# NAME : PRAJITHA MARY.J
# SpendDNA - Industry Graded Minor Project

# FEATURE - 1

# Import libraries

import pandas as pd
import numpy as np

# Load the dataset

df = pd.read_csv("Data set for DADS June.csv")

df.head()

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 1328,\n  \"fields\": [\n    {\n      \"column\": \"Date\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 617,\n        \"samples\": [\n          \"15-Jan-24\",\n          \"21 Jun 2024\",\n          \"26/01/24\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Time\",\n      \"properties\": {\n        \"dtype\": \"object\",\n        \"num_unique_values\": 832,\n        \"samples\": [\n          \"10:40\",\n          \"05:53\",\n          \"12:12\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Description\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 283,\n        \"samples\": [\n          \"OLA ELECTRIC\",\n          \"UPI-ANKIT-5946@OKAXIS\",\n          \"UPI-RESTAURANT-0251@PAYTM\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Type\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"Debit\",\n          \"DR\",\n          \"CR\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Amount\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1141,\n        \"samples\": [\n          \"590\",\n          \"625.00\",\n          \"Rs. 489\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Balance\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 341059.80996944895,\n        \"min\": -769127.0,\n        \"max\": 720079.0,\n        \"num_unique_values\": 1310,\n        \"samples\": [\n          133249.0,\n          -88674.0,\n          411272.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Mode\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"NEFT\",\n          \"ATM\",\n          \"UPI\"\n        ],\n        \"semantic_type\": \"\",

```

```
df.tail()
```

```
print("SHAPE:\n",df.shape)
print("\n")
print("COLUMN:\n",df.columns)
print("\n")
print("DATA TYPES:\n",df.dtypes)
print("\n")
print("INFORMATION:\n",df.info())
print("\n")
print("MISSING VALUES:\n",df.isnull().sum())
```

```
COLUMN:
  Index(['Date', 'Time', 'Description', 'Type', 'Amount', 'Balance',
        'Mode',
        'Ref'],
        dtype='object')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1328 entries, 0 to 1327
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Date            1328 non-null   object
 1   Time            1328 non-null   object
```

```
2  Description  1328 non-null  object
3  Type        1328 non-null  object
4  Amount      1328 non-null  object
5  Balance     1328 non-null  float64
6  Mode        1328 non-null  object
7  Ref         1328 non-null  object
```

```
dtypes: float64(1), object(7)
```

```
memory usage: 83.1+ KB
```

```
INFORMATION:
```

```
None
```

```
MISSING VALUES:
```

```
  Date      0
  Time      0
  Description 0
  Type      0
  Amount    0
  Balance    0
  Mode      0
  Ref       0
```

```
dtype: int64
```

```
# Count duplicate rows
```

```
print("DUPLICATE ROWS:",df.duplicated().sum())
```

```
# Remove duplicates
```

```
df = df.drop_duplicates()
```

```
print("AFTER REMOVING DUPLICATES:",df.shape)
```

```
DUPLICATE ROWS: 18
```

```
AFTER REMOVING DUPLICATES: (1310, 8)
```

```
# Convert Date column into datetime format
```

```
df["Date"] = pd.to_datetime(
    df["Date"],
    errors="coerce",
    dayfirst=True
)
```

```
# invalid dates
```

```
print("INVALID DATES:", df["Date"].isnull().sum())
```

```
INVALID DATES: 1167
```

```
# Convert Amount column into string
```

```

df["Amount"] = df["Amount"].astype(str)
# Remove Rupee Symbol
df["Amount"] = df["Amount"].str.replace("₹", "", regex=False)
# Remove Rs.
df["Amount"] = df["Amount"].str.replace("Rs.", "", regex=False)
# Remove commas
df["Amount"] = df["Amount"].str.replace(",", "", regex=False)
# Remove extra spaces
df["Amount"] = df["Amount"].str.strip()
# Convert into numeric
df["Amount"] = pd.to_numeric(
    df["Amount"],
    errors="coerce"
)
print("INVALID AMOUNTS:", df["Amount"].isnull().sum())
INVALID AMOUNTS: 0
# Convert Type column into lowercase
df["Type"] = df["Type"].str.lower()
# Replace values
df["Type"] = df["Type"].replace({
    "dr": "debit",
    "cr": "credit"
})
print(df["Type"].value_counts())
Type
debit    1304
credit      6
Name: count, dtype: int64
# Replace empty values with NaN
df["Mode"] = df["Mode"].replace("", np.nan)
# Check missing values

```

```
print("MISSING VALUE:",df["Mode"].isnull().sum())
```

```
MISSING VALUE: 0
```

```
# Remove rows having invalid Date or Amount
```

```
df = df.dropna(subset=["Date","Amount"])
```

```
print("DATASET SHAPE:",df.shape)
```

```
DATASET SHAPE: (143, 8)
```

```
print("INFO:",df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 143 entries, 0 to 1197
```

```
Data columns (total 8 columns):
```

#	Column	Non-Null Count	Dtype
0	Date	143 non-null	datetime64[ns]
1	Time	143 non-null	object
2	Description	143 non-null	object
3	Type	143 non-null	object
4	Amount	143 non-null	float64
5	Balance	143 non-null	float64
6	Mode	143 non-null	object
7	Ref	143 non-null	object

```
dtypes: datetime64[ns](1), float64(2), object(5)
```

```
memory usage: 10.1+ KB
```

```
INFO: None
```

```
df.head()
```

```
{
  "summary": {
    "name": "df",
    "rows": 143,
    "fields": [
      {
        "column": "Date",
        "properties": {
          "dtype": "date",
          "min": "2024-01-01 00:00:00",
          "max": "2024-12-06 00:00:00",
          "num_unique_values": 63,
          "samples": [
            "2024-11-06 00:00:00",
            "2024-07-06 00:00:00",
            "2024-01-01 00:00:00"
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Time",
        "properties": {
          "dtype": "object",
          "num_unique_values": 136,
          "samples": [
            "12:33",
            "09:35",
            "14:09"
          ],
          "semantic_type": ""
        },
        "description": ""
      },
      {
        "column": "Description",
        "properties": {
          "dtype": "string",
          "num_unique_values": 81,
          "samples": [
            "UPI-SWIGGY4619@0KAXIS",
            "AMAZON SELLER SVCS",
            "UPI-SWIGGY-7959@HDFCBANK"
          ],
          "semantic_type": ""
        },
        "description": ""
      }
    ]
  }
}
```

```

{"column": "Type", "properties": {"dtype": "category", "num_unique_values": 2, "samples": [{"credit", "debit"}], "semantic_type": "", "description": ""}, {"column": "Amount", "properties": {"dtype": "number", "std": 10378.375386824822, "min": 35.0, "max": 85393.0, "num_unique_values": 132, "samples": [3000.0, 283.0]}, {"column": "Balance", "properties": {"dtype": "number", "std": 355446.8269815285, "min": -769127.0, "max": 702079.0, "num_unique_values": 143, "samples": [-40620.0, 634839.0]}, {"column": "Mode", "properties": {"dtype": "category", "num_unique_values": 4, "samples": ["NEFT", "ATM"]}, {"column": "Ref", "properties": {"dtype": "string", "num_unique_values": 143, "samples": ["TXN301998", "TXN248298"]}, {"column": "Description", "properties": {"dtype": "string", "num_unique_values": 143, "samples": ["TXN301998", "TXN248298"]}}], "type": "dataframe", "variable_name": "df"}

```

```
df.tail()
```

```
{"repr_error": "0", "type": "dataframe"}
```

```
# Feature 1 Output
```

```
print("="*60)
```

```
print("FEATURE 1 - TRANSACTION PARSE")
```

```
print("="*60)
```

```
print("TOTAL TRANSACTION : ", len(df))
```

```
print("DUPLICATE ROWS REMOVED : ", df.duplicated().sum())
```

```
print("INVALID DATES : ", df["Date"].isnull().sum())
```

```
print("INVALID AMOUNTS : ", df["Amount"].isnull().sum())
```

```
print("DATA RANGE:\n", df["Date"].min())
```

```
print("to")
```

```
print(df["Date"].max())
```

```
print("="*60)
```

```
=====
FEATURE 1 - TRANSACTION PARSE
=====
```

```
TOTAL TRANSACTION : 143
```

```
DUPLICATE ROWS REMOVED      : 0
INVALID DATES                : 0
INVALID AMOUNTS              : 0
```

```
DATA RANGE:
```

```
2024-01-01 00:00:00
```

```
to
```

```
2024-12-06 00:00:00
```

```
=====
```

```
# FEATURE - 2 -----> Vendor Extractor
```

```
# unique transaction descriptions
```

```
print("UNIQUE TRANSACTION:\n",df["Description"].unique()[ :50])
```

```
UNIQUE TRANSACTION:
```

```
['AMAZON SELLER SVCS' 'UPI-AMAN-8934@OKAXIS' 'BHIM ZEPTO'
'UPI-UBER-2426@HDFCBANK' 'POS SWIGGY BANGALORE' 'ANI Technologies'
'POS SWIGGY-RESTAURANT' 'POS UBER BANGALORE' 'TWC INDIA'
'UPI-STARBUCKS@AXIS' 'POS BLINKIT' 'ZOMATO MEDIA P L' 'BHIM SWIGGY'
'OLA CABS' 'UPI-DMART@OKAXIS' 'POS ZEPTO BANGALORE' 'UPI-
MYNTRA@HDFCBANK'
'BANGALORE ELEC SUPPLY' 'Amazon Pay India' 'UPI-PRIYA-2221@OKAXIS'
'INSTAMART BANGALORE' 'UPI-ZOMATO-LIMITED@PAYTM'
'UPI-SWIGGY-7959@HDFCBANK' 'UPI-SWIGGY-INSTAMART@OKAXIS' 'POS OLA-
PRIME'
'POS EMPIRE RESTAURANT' 'SWIGGY-INSTAMART' 'POS NYKAA BANGALORE'
'ROPEN TRANSPORTATION' 'UPI-NYKAA@AXIS' 'UPI-SWIGGY4619@OKAXIS'
'UPI-FLIPKART@HDFCBANK' 'IMPS ZERODHA-COIN' 'UPI-THIRDWAVE@OKAXIS'
'UPI-SWIGGY-1309@HDFCBANK' 'NEFT-TECHCRUSH LABS-SALARY MAY24'
'BHIM-BLINKIT' 'UPI-ZEPTO-4547@HDFCBANK' 'RAPIDO BIKE TAXI'
'UPI-RESTAURANT-6377@PAYTM' 'Swiggy*Order' 'COFFEE DAY GLOBAL'
'FLIPKART INDIA' 'UPI-CCD@HDFCBANK' 'UPI-VIKAS-5416@OKAXIS'
'UPI-BOOKMYSHOW@HDFCBANK' 'UPI-HOTSTAR@AXIS' 'UPI-ZOMATO6436@OKICICI'
'NETFLIX.COM' 'BMS MOVIE TICKETS']
```

```
# Dictionary for vendor mapping
```

```
vendor_dict = {
    "Swiggy": ["SWIGGY", "BUNDL"],
    "Zomato": ["ZOMATO"],
    "Zepto": ["ZEPTO"],
    "Blinkit": ["BLINKIT"],
    "BigBasket": ["BIGBASKET"],
    "Amazon": ["AMAZON", "AMZN", "AMAZONPAY"],
    "Flipkart": ["FLIPKART"],
    "Myntra": ["MYNTRA"],
    "Ajio": ["AJIO"],
    "Nykaa": ["NYKAA"],
```

```

"BookMyShow": ["BOOKMYSHOW"],
"Netflix": ["NETFLIX"],
"Spotify": ["SPOTIFY"],
"Hotstar": ["HOTSTAR"],
"Prime Video": ["PRIMEVIDEO", "PRIME VIDEO"],
"YouTube": ["YOUTUBE"],
"Uber": ["UBER"],
"Ola": ["OLA"],
"Rapido": ["RAPIDO"],
"Metro": ["METRO"],
"IRCTC": ["IRCTC"],
"Indian Oil": ["INDIANOIL", "IOCL"],
"HP": ["HPCL"],
"Bharat Petroleum": ["BPCL"],
"Reliance Fresh": ["RELIANCE"],
"DMart": ["DMART"],
"Starbucks": ["STARBUCKS"],
"Cafe Coffee Day": ["CCD"],
"Third Wave Coffee": ["THIRD WAVE"],
"Zerodha": ["ZERODHA"],
"Groww": ["GROWW"],
"PhonePe": ["PHONEPE"],
"Google Pay": ["GPAY", "GOOGLEPAY"],
"Paytm": ["PAYTM"],
"Electricity": ["BESCOM"],
"Water Bill": ["BWSSB"],
"Gas": ["GAS"],
"ATM Withdrawal": ["ATM"],
"P2P Transfer": ["PRIYA", "ANKIT", "RAHUL", "ROHIT", "SNEHA",
"AMIT", "NEHA"]
}

# Function to identify vendor

def extract_vendor(description):
    description = str(description).upper()
    for vendor in vendor_dict:
        keywords = vendor_dict[vendor]
        for word in keywords:
            if word in description:
                return vendor

    return "Uncategorised"

# Create a new column

df["vendor_clean"] = df["Description"].apply(extract_vendor)

# Check Extracted Vendors

```

```

print("TOTAL UNIQUE VENDORS:\n",df["vendor_clean"].nunique())
print("VENDOR COUNTS:",df["vendor_clean"].value_counts())

TOTAL UNIQUE VENDORS:
25
VENDOR COUNTS: vendor_clean
Uncategorised      34
Swiggy              28
Zomato              16
Ola                 8
Uber                7
Zepto               5
Starbucks           4
Flipkart            4
Nykaa               4
Zerodha             4
Amazon              3
ATM Withdrawal      3
DMart               3
BookMyShow          3
Blinkit             2
P2P Transfer        2
Myntra              2
Paytm               2
Rapido              2
Cafe Coffee Day     2
Hotstar             1
Netflix             1
Third Wave Coffee   1
BigBasket           1
Groww               1
Name: count, dtype: int64

# top 10 vendors

print("TOP 10 VENDORS:",df["vendor_clean"].value_counts().head(10))

TOP 10 VENDORS: vendor_clean
Uncategorised      34
Swiggy              28
Zomato              16
Ola                 8
Uber                7
Zepto               5
Starbucks           4
Flipkart            4
Nykaa               4
Zerodha             4
Name: count, dtype: int64

```

```
# Check Uncategorized Vendors
```

```
uncategorised = df[df["vendor_clean"] == "Uncategorised"]
```

```
print("NUMBER OF UNCATEGORISED TRANSACTION:")
```

```
print(len(uncategorised))
```

```
print("DESCRIPTION:")
```

```
print(uncategorised["Description"].unique())
```

```
NUMBER OF UNCATEGORISED TRANSACTION:
```

```
34
```

```
DESCRIPTION:
```

```
['UPI-AMAN-8934@OKAXIS' 'ANI Technologies' 'TWC INDIA'  
'BANGALORE ELEC SUPPLY' 'INSTAMART BANGALORE' 'POS EMPIRE RESTAURANT'  
'ROPPEN TRANSPORTATION' 'UPI-THIRDWAVE@OKAXIS'  
'NEFT-TECHCRUSH LABS-SALARY MAY24' 'COFFEE DAY GLOBAL'  
'UPI-VIKAS-5416@OKAXIS' 'BMS MOVIE TICKETS' 'INNOVATIVE RETAIL'  
'AVENUE SUPERMARTS' 'POS HP PETROL STATION' 'KIRANAKART TECH'  
'POS BANGALORE RESTAURANT' 'IMPS-RENT-LANDLORD-49966195' 'POS  
INSTAMART'  
'IMPS-RENT-LANDLORD-35126704' 'UPI-VI-RECHARGE@HDFCBANK']
```

```
# Feature 2 Output
```

```
print("=" * 60)
```

```
print("                FEATURE 2 - VENDOR EXTRACTOR")
```

```
print("=" * 60)
```

```
print("TOTAL VENDORS IDENTIFIED:", df["vendor_clean"].nunique())
```

```
print()
```

```
print("TOP 10 VENDORS:")
```

```
print(df["vendor_clean"].value_counts().head(10))
```

```
print()
```

```
print("UNCATEGORISED TRANSACTION:", len(uncategorised))
```

```
print("=" * 60)
```

```
=====
```

```
                FEATURE 2 - VENDOR EXTRACTOR
```

```
=====
```

```
TOTAL VENDORS IDENTIFIED: 25
```

```
TOP 10 VENDORS:
```

```
vendor_clean
Uncategorised      34
Swiggy              28
Zomato              16
Ola                 8
Uber                7
Zepto               5
Starbucks           4
Flipkart            4
Nykaa               4
Zerodha             4
Name: count, dtype: int64
```

```
UNCATEGORISED TRANSACTION: 34
```

```
=====
```

```
# FEATURES - 3 -----> Category Tagger
```

```
# Dictionary to map vendors to categories
```

```
category_dict = {
```

```
    "Swiggy": "Food Delivery",
    "Zomato": "Food Delivery",
```

```
    "Zepto": "Quick Commerce",
    "Blinkit": "Quick Commerce",
```

```
    "BigBasket": "Groceries",
    "Reliance Fresh": "Groceries",
    "DMart": "Groceries",
```

```
    "Amazon": "E-commerce",
    "Flipkart": "E-commerce",
    "Myntra": "E-commerce",
    "Ajio": "E-commerce",
    "Nykaa": "E-commerce",
```

```
    "BookMyShow": "Entertainment",
```

```
    "Netflix": "Subscriptions",
    "Spotify": "Subscriptions",
    "Prime Video": "Subscriptions",
    "Hotstar": "Subscriptions",
    "YouTube": "Subscriptions",
```

```
    "Uber": "Transport",
    "Ola": "Transport",
    "Rapido": "Transport",
    "Metro": "Transport",
```

```

    "IRCTC": "Transport",

    "Starbucks": "Cafe",
    "Cafe Coffee Day": "Cafe",
    "Third Wave Coffee": "Cafe",

    "Zerodha": "Investments",
    "Groww": "Investments",

    "Electricity": "Utilities",
    "Water Bill": "Utilities",
    "Gas": "Utilities",

    "Indian Oil": "Fuel",
    "HP": "Fuel",
    "Bharat Petroleum": "Fuel",

    "P2P Transfer": "Personal Transfer",

    "ATM Withdrawal": "Cash Withdrawal"
}

# Create category column
df["category"] = df["vendor_clean"].map(category_dict)

# Replace missing values
df["category"] = df["category"].fillna("Others")

# Display Category Counts
print("CATEGORY COUNTS:\n",df["category"].value_counts())

CATEGORY COUNTS:
category
Food Delivery      44
Others              36
Transport           17
E-commerce          13
Quick Commerce      7
Cafe                7
Investments         5
Groceries           4
Cash Withdrawal     3
Entertainment       3
Personal Transfer   2
Subscriptions        2
Name: count, dtype: int64

```

```
# Check Missing Categories
```

```
print("MISSING CATEGORIES:",df["category"].isnull().sum())
```

```
MISSING CATEGORIES: 0
```

```
# Display Top Categories
```

```
print("TOP CATEGORIES:\n",df["category"].value_counts().head())
```

```
TOP CATEGORIES:
```

```
category
Food Delivery    44
Others            36
Transport        17
E-commerce       13
Quick Commerce   7
Name: count, dtype: int64
```

```
# Verify Mapping
```

```
print(df[["vendor_clean","category"]].head(20))
```

	vendor_clean	category
0	Amazon	E-commerce
3	Uncategorised	Others
5	Zepto	Quick Commerce
6	Uber	Transport
8	Swiggy	Food Delivery
16	Uncategorised	Others
20	Swiggy	Food Delivery
22	Uber	Transport
23	Swiggy	Food Delivery
25	Uncategorised	Others
29	Starbucks	Cafe
33	Blinkit	Quick Commerce
35	Zomato	Food Delivery
51	Swiggy	Food Delivery
54	Ola	Transport
58	DMart	Groceries
62	Uber	Transport
68	Zepto	Quick Commerce
70	Myntra	E-commerce
71	Uber	Transport

```
# Category-wise Transactions
```

```
print("CATEGORY_COUNT:\n",df.groupby("category").size())
```

```
CATEGORY_COUNT:
```

```
category
```

Cafe	7
Cash Withdrawal	3
E-commerce	13
Entertainment	3
Food Delivery	44
Groceries	4
Investments	5
Others	36
Personal Transfer	2
Quick Commerce	7
Subscriptions	2
Transport	17

dtype: int64

Category-wise Total Amount

```
print("CATEGORY_AMOUNT:", df.groupby("category")["Amount"].sum())
```

```
CATEGORY_AMOUNT: category
Cafe                2434.0
Cash Withdrawal     3000.0
E-commerce          36416.0
Entertainment       1740.0
Food Delivery       19617.0
Groceries           5242.0
Investments         64496.0
Others              231025.0
Personal Transfer   1512.0
Quick Commerce      3094.0
Subscriptions       1683.0
Transport           5145.0
Name: Amount, dtype: float64
```

Feature 3 Output

```
print("="*60)
print("                FEATURE 3 - CATEGORY TAGGER")
print("="*60)

print("TOTAL CATEGORIES:", df["category"].nunique())

print()

print("CATEGORY COUNTS:\n", df["category"].value_counts())

print()

print("TRANSACTIONS SUCCESSFULLY CATEGORISED")

print(len(df))
```

```
print("="*60)
```

```
=====
                        FEATURE 3 - CATEGORY TAGGER
=====
```

```
TOTAL CATEGORIES: 12
```

```
CATEGORY COUNTS:
```

```
category
Food Delivery      44
Others              36
Transport           17
E-commerce          13
Quick Commerce      7
Cafe                7
Investments          5
Groceries           4
Cash Withdrawal     3
Entertainment        3
Personal Transfer    2
Subscriptions        2
Name: count, dtype: int64
```

```
TRANSACTIONS SUCCESSFULLY CATEGORISED
143
```

```
=====
# FEATURE - 4 -----> Spending Overview
```

```
# Create separate DataFrames
```

```
credit_df = df[df["Type"] == "credit"]
debit_df = df[df["Type"] == "debit"]
```

```
# Calculate total credit and debit amount
```

```
total_credit = credit_df["Amount"].sum()
total_debit = debit_df["Amount"].sum()
```

```
print("TOTAL CREDIT:",total_credit)
print("TOTAL DEBIT:",total_debit)
```

```
TOTAL CREDIT: 170094.0
TOTAL DEBIT: 205310.0
```

```
# Net Savings
```

```
net_savings = total_credit - total_debit
```

```
print("NET SAVINGS:",net_savings)
```

NET SAVINGS: -35216.0

Savings Rate

savings_rate = (net_savings / total_credit) * 100

print("SAVING RATE:", round(savings_rate, 2), "%")

SAVING RATE: -20.7 %

Total Transactions

total_transactions = len(df)

print("TOTAL TRANSACTIONS:", total_transactions)

TOTAL TRANSACTIONS: 143

Total Unique Vendors

unique_vendors = df["vendor_clean"].nunique()

print("UNIQUE VENDORS:", df["vendor_clean"].nunique())

UNIQUE VENDORS: 25

Category-wise spending

top_categories = debit_df.groupby("category")["Amount"].sum()

top_categories = top_categories.sort_values(ascending=False)

print(top_categories.head())

category	
Investments	64496.0
Others	60931.0
E-commerce	36416.0
Food Delivery	19617.0
Groceries	5242.0

Name: Amount, dtype: float64

Vendor-wise spending

top_vendors = debit_df.groupby("vendor_clean")["Amount"].sum()

top_vendors = top_vendors.sort_values(ascending=False)

print(top_vendors.head())

vendor_clean	
Zerodha	60000.0
Uncategorised	59831.0

```
Myntra          13629.0
Swiggy          12638.0
Flipkart       10041.0
Name: Amount, dtype: float64
```

```
vendor_transactions = debit_df["vendor_clean"].value_counts()
```

```
print(vendor_transactions.head())
```

```
vendor_clean
Uncategorised    32
Swiggy           28
Zomato           16
Ola              8
Uber             7
Name: count, dtype: int64
```

```
category_percentage = (top_categories / total_debit) * 100
```

```
category_percentage = category_percentage.round(2)
```

```
print(category_percentage.head())
```

```
category
Investments    31.41
Others         29.68
E-commerce     17.74
Food Delivery   9.55
Groceries       2.55
Name: Amount, dtype: float64
```

```
print("="*65)
```

```
print("EXECUTIVE SUMMARY")
```

```
print("="*65)
```

```
print(f"Total Credits      : ₹ {total_credit:.2f}")
```

```
print(f"Total Debits       : ₹ {total_debit:.2f}")
```

```
print(f"Net Savings          : ₹ {net_savings:.2f}")
```

```
print(f"Savings Rate         : {savings_rate:.2f}%")
```

```
print(f"Total Transactions    : {total_transactions}")
```

```
print(f"Unique Vendors        : {unique_vendors}")
```

```
print("="*65)
```

```
=====
EXECUTIVE SUMMARY
=====
```

```
Total Credits      : ₹ 170094.00
Total Debits       : ₹ 205310.00
Net Savings        : ₹ -35216.00
Savings Rate       : -20.70%
Total Transactions  : 143
Unique Vendors     : 25
```

```
print()

print("="*65)
print("                TOP 5 SPENDING CATEGORIES")
print("="*65)

for category, amount in top_categories.head().items():
    percentage = (amount / total_debit) * 100
    print(f"{category:<25} ₹ {amount:>10.2f}    ({percentage:.2f}%)")
```

```
=====
                        TOP 5 SPENDING CATEGORIES
=====
```

Investments	₹	64496.00	(31.41%)
Others	₹	60931.00	(29.68%)
E-commerce	₹	36416.00	(17.74%)
Food Delivery	₹	19617.00	(9.55%)
Groceries	₹	5242.00	(2.55%)

```
print()

print("="*65)
print("                TOP 5 VENDORS")
print("="*65)

for vendor, amount in top_vendors.head().items():
    count = len(debit_df[debit_df["vendor_clean"] == vendor])
    print(f"{vendor:<20} ₹ {amount:>10.2f}    ({count} Transactions)")
```

```
=====
                        TOP 5 VENDORS
=====
```

Zerodha	₹	60000.00	(4 Transactions)
Uncategorised	₹	59831.00	(32 Transactions)
Myntra	₹	13629.00	(2 Transactions)
Swiggy	₹	12638.00	(28 Transactions)
Flipkart	₹	10041.00	(4 Transactions)

```

print()

print("="*65)
print("                                FEATURE 4 COMPLETED")
print("="*65)

print("✓ Spending Overview Generated Successfully")
print("✓ Total Credits Calculated")
print("✓ Total Debits Calculated")
print("✓ Savings Rate Calculated")
print("✓ Top Categories Identified")
print("✓ Top Vendors Identified")
print("="*65)

```

```

=====
                                FEATURE 4 COMPLETED
=====
✓ Spending Overview Generated Successfully
✓ Total Credits Calculated
✓ Total Debits Calculated
✓ Savings Rate Calculated
✓ Top Categories Identified
✓ Top Vendors Identified
=====

```

FEATURE - 5 -----> Monthly Trend Analysis

Create month number column

```
df["month"] = df["Date"].dt.month
```

Create month name column

```
df["month_name"] = df["Date"].dt.strftime("%b")
```

```
print(df[["Date", "month", "month_name"]].head())
```

	Date	month	month_name
0	2024-01-01	1	Jan
3	2024-01-01	1	Jan
5	2024-01-01	1	Jan
6	2024-01-01	1	Jan
8	2024-02-01	2	Feb

```
# Use only debit transactions

monthly_df = df[df["Type"] == "debit"]

# Create Pivot Table

month_pivot = monthly_df.pivot_table(
    values="Amount",
    index="category",
    columns="month_name",
    aggfunc="sum",
    fill_value=0
)

print(month_pivot)
```

month_name category	Apr	Aug	Dec	Feb	Jan	Jul
Cafe	447.0	362.0	0.0	355.0	295.0	0.0
Cash Withdrawal	0.0	0.0	500.0	0.0	0.0	0.0
E-commerce	3311.0	860.0	4092.0	996.0	6972.0	2425.0
Entertainment	343.0	0.0	0.0	0.0	0.0	562.0
Food Delivery	632.0	1446.0	2758.0	3454.0	1180.0	1686.0
Groceries	0.0	1862.0	910.0	0.0	0.0	1257.0
Investments	0.0	34496.0	15000.0	0.0	0.0	15000.0
Others	513.0	1991.0	6219.0	267.0	4874.0	682.0
Personal Transfer	0.0	0.0	1266.0	0.0	0.0	0.0
Quick Commerce	0.0	0.0	0.0	697.0	825.0	196.0
Subscriptions	0.0	0.0	0.0	0.0	0.0	1473.0
Transport	0.0	366.0	0.0	621.0	826.0	351.0
month_name category	Mar	May	Nov	Oct	Sep	
Cafe	0.0	166.0	809.0	0.0	0.0	
Cash Withdrawal	0.0	0.0	2500.0	0.0	0.0	
E-commerce	2147.0	0.0	0.0	0.0	10745.0	

Entertainment	0.0	0.0	0.0	835.0	0.0
Food Delivery	2862.0	1159.0	1909.0	183.0	712.0
Groceries	1213.0	0.0	0.0	0.0	0.0
Investments	0.0	0.0	0.0	0.0	0.0
Others	19917.0	18753.0	766.0	3291.0	1004.0
Personal Transfer	0.0	0.0	0.0	0.0	246.0
Quick Commerce	0.0	502.0	347.0	0.0	527.0
Subscriptions	0.0	0.0	0.0	0.0	210.0
Transport	439.0	431.0	35.0	1232.0	710.0

Arrange months in order

```
month_order = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
```

```
month_pivot = month_pivot.reindex(columns=month_order)
```

```
print(month_pivot)
```

month_name	Jan	Feb	Mar	Apr	May	Jun
category						
Cafe	295.0	355.0	0.0	447.0	166.0	0.0
Cash Withdrawal	0.0	0.0	0.0	0.0	0.0	0.0
E-commerce	6972.0	996.0	2147.0	3311.0	0.0	4868.0
Entertainment	0.0	0.0	0.0	343.0	0.0	0.0
Food Delivery	1180.0	3454.0	2862.0	632.0	1159.0	1636.0
Groceries	0.0	0.0	1213.0	0.0	0.0	0.0
Investments	0.0	0.0	0.0	0.0	0.0	0.0
Others	4874.0	267.0	19917.0	513.0	18753.0	2654.0
Personal Transfer	0.0	0.0	0.0	0.0	0.0	0.0
Quick Commerce	825.0	697.0	0.0	0.0	502.0	0.0
Subscriptions	0.0	0.0	0.0	0.0	0.0	0.0
Transport	826.0	621.0	439.0	0.0	431.0	134.0

Calculate percentage growth

```
growth = ((month_pivot["Jun"] - month_pivot["Jan"]) /
           month_pivot["Jan"]) * 100
```

```
growth = growth.replace([np.inf, -np.inf], 0)
```

```
growth = growth.fillna(0)
```

```
print(growth)
```

category	
Cafe	-100.000000
Cash Withdrawal	0.000000
E-commerce	-30.177854
Entertainment	0.000000
Food Delivery	38.644068
Groceries	0.000000

```
Investments      0.000000
Others           -45.547805
Personal Transfer 0.000000
Quick Commerce  -100.000000
Subscriptions    0.000000
Transport        -83.777240
dtype: float64
```

```
highest_growth = growth.idxmax()
```

```
highest_growth_value = growth.max()
```

```
print("HIGHEST GROWTH CATEGORY")
```

```
print(highest_growth)
```

```
print(round(highest_growth_value,2), "%")
```

```
HIGHEST GROWTH CATEGORY
```

```
Food Delivery
```

```
38.64 %
```

```
highest_decline = growth.idxmin()
```

```
highest_decline_value = growth.min()
```

```
print("HIGHEST DECLINE CATEGORY")
```

```
print(highest_decline)
```

```
print(round(highest_decline_value,2), "%")
```

```
HIGHEST DECLINE CATEGORY
```

```
Cafe
```

```
-100.0 %
```

```
monthly_total = monthly_df.groupby("month_name")["Amount"].sum()
```

```
monthly_total = monthly_total.reindex(month_order)
```

```
print(monthly_total)
```

```
month_name
```

```
Jan    14972.0
```

```
Feb     6390.0
```

```
Mar    26578.0
```

```
Apr     5246.0
```

```
May    21011.0
```

```
Jun     9292.0
```

```
Name: Amount, dtype: float64
```

```
print("="*65)
print("                MONTHLY SPENDING TREND")
print("="*65)
```

```
for month, amount in monthly_total.items():
```

```
    print(f"{month:<5} ₹ {amount:.2f}")
```

```
=====
                MONTHLY SPENDING TREND
=====
```

```
Jan    ₹ 14972.00
Feb    ₹ 6390.00
Mar    ₹ 26578.00
Apr    ₹ 5246.00
May    ₹ 21011.00
Jun    ₹ 9292.00
```

```
print("="*65)
print("                CATEGORY WISE MONTHLY SPENDING")
print("="*65)
```

```
print(month_pivot)
```

```
=====
                CATEGORY WISE MONTHLY SPENDING
=====
```

month_name	Jan	Feb	Mar	Apr	May	Jun
category						
Cafe	295.0	355.0	0.0	447.0	166.0	0.0
Cash Withdrawal	0.0	0.0	0.0	0.0	0.0	0.0
E-commerce	6972.0	996.0	2147.0	3311.0	0.0	4868.0
Entertainment	0.0	0.0	0.0	343.0	0.0	0.0
Food Delivery	1180.0	3454.0	2862.0	632.0	1159.0	1636.0
Groceries	0.0	0.0	1213.0	0.0	0.0	0.0
Investments	0.0	0.0	0.0	0.0	0.0	0.0
Others	4874.0	267.0	19917.0	513.0	18753.0	2654.0
Personal Transfer	0.0	0.0	0.0	0.0	0.0	0.0
Quick Commerce	825.0	697.0	0.0	0.0	502.0	0.0
Subscriptions	0.0	0.0	0.0	0.0	0.0	0.0
Transport	826.0	621.0	439.0	0.0	431.0	134.0

```
print("="*65)
print("                MONTHLY SPENDING BAR")
print("="*65)
```

```
max_amount = monthly_total.max()
```

```
for month, amount in monthly_total.items():
```

```
    bar_length = int((amount / max_amount) * 30)
```

```
bar = "#" * bar_length

print(f"{month:<5} {bar} ₹ {amount:.2f}")
```

```
=====
                        MONTHLY SPENDING BAR
=====
Jan ##### ₹ 14972.00
Feb ##### ₹ 6390.00
Mar ##### ₹ 26578.00
Apr ##### ₹ 5246.00
May ##### ₹ 21011.00
Jun ##### ₹ 9292.00
```

```
growth_table = pd.DataFrame({
    "Category": growth.index,
    "Growth (%)": growth.values
})

growth_table = growth_table.sort_values(
    by="Growth (%)",
    ascending=False
)
```

```
print(growth_table)
```

	Category	Growth (%)
4	Food Delivery	38.644068
1	Cash Withdrawal	0.000000
5	Groceries	0.000000
3	Entertainment	0.000000
10	Subscriptions	0.000000
8	Personal Transfer	0.000000
6	Investments	0.000000
2	E-commerce	-30.177854
7	Others	-45.547805
11	Transport	-83.777240
0	Cafe	-100.000000
9	Quick Commerce	-100.000000

```
print("="*65)
print("                        FEATURE 5 - MONTHLY TREND ANALYSIS")
print("="*65)
```

```

print("HIGHEST GROWTH CATEGORY")
print(highest_growth)
print()
print("GROWTH PERCENTAGE")
print(round(highest_growth_value,2), "%")
print()
print("HIGHEST DECLINE CATEGORY")
print(highest_decline)
print()
print("DECLINE PERCENTAGE")
print(round(highest_decline_value,2), "%")
print("="*65)

```

```

=====
FEATURE 5 - MONTHLY TREND ANALYSIS
=====

```

```

HIGHEST GROWTH CATEGORY
Food Delivery

```

```

GROWTH PERCENTAGE
38.64 %

```

```

HIGHEST DECLINE CATEGORY
Cafe

```

```

DECLINE PERCENTAGE
-100.0 %
=====

```

```

# FEATURE - 6 -----> Time-of-Day Patterns

```

```

# Extract hour from Time column

```

```

df["hour"] = df["Time"].str[:2].astype(int)

```

```

# Display first few rows

```

```

print(df[["Time", "hour"]].head())

```

```

    Time  hour
0  03:11     3

```

3	14:07	14
5	14:48	14
6	14:50	14
8	05:18	5

Function to classify time of the day

```
def time_period(hour):
    if 5 <= hour < 12:
        return "Morning"

    elif 12 <= hour < 17:
        return "Afternoon"

    elif 17 <= hour < 21:
        return "Evening"

    else:
        return "Night"
```

Apply function

```
df["time_period"] = df["hour"].apply(time_period)
print(df[["Time", "hour", "time_period"]].head())
```

	Time	hour	time_period
0	03:11	3	Night
3	14:07	14	Afternoon
5	14:48	14	Afternoon
6	14:50	14	Afternoon
8	05:18	5	Morning

```
print("="*60)
print("                TRANSACTIONS BY TIME PERIOD")
print("="*60)
```

```
print(df["time_period"].value_counts())
```

```
=====
                TRANSACTIONS BY TIME PERIOD
=====
time_period
Afternoon    41
Morning      39
Evening      33
Night        30
Name: count, dtype: int64
```

```
time_spending = df[df["Type"]=="debit"].groupby("time_period")
["Amount"].sum()
```

```
print("="*60)
print("                                SPENDING BY TIME PERIOD")
print("="*60)
```

```
print(time_spending)
```

```
=====
                                SPENDING BY TIME PERIOD
=====
```

```
time_period
Afternoon    56789.0
Evening      39460.0
Morning      73607.0
Night        35454.0
Name: Amount, dtype: float64
```

```
hour_matrix = df[df["Type"]=="debit"].pivot_table(
    values="Amount",
    index="category",
    columns="hour",
    aggfunc="sum",
    fill_value=0
)
```

```
print(hour_matrix)
```

hour \ category	0	1	2	3	4	5	6
Cafe	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Cash Withdrawal	0.0	0.0	0.0	0.0	0.0	0.0	0.0
E-commerce	0.0	842.0	0.0	2462.0	0.0	0.0	0.0
Entertainment	562.0	0.0	0.0	0.0	0.0	0.0	0.0
Food Delivery	394.0	0.0	318.0	0.0	413.0	537.0	0.0
Groceries	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Investments	0.0	0.0	0.0	0.0	15000.0	4496.0	
0.0							
Others	433.0	620.0	0.0	766.0	0.0	0.0	
0.0							
Personal Transfer	0.0	0.0	0.0	0.0	0.0	0.0	
0.0							
Quick Commerce	0.0	0.0	697.0	0.0	0.0	0.0	
0.0							
Subscriptions	0.0	0.0	0.0	0.0	0.0	0.0	
0.0							
Transport	431.0	0.0	0.0	0.0	0.0	425.0	
322.0							
hour	7	8	9	...	14	15	16
17 \							
category				...			
Cafe	0.0	0.0	376.0	...	166.0	0.0	447.0
295.0							
Cash Withdrawal	0.0	0.0	500.0	...	500.0	0.0	0.0
0.0							
E-commerce	860.0	0.0	0.0	...	0.0	1583.0	0.0
0.0							
Entertainment	0.0	0.0	0.0	...	0.0	0.0	835.0
0.0							
Food Delivery	0.0	369.0	383.0	...	368.0	1274.0	377.0
919.0							
Groceries	0.0	0.0	0.0	...	0.0	0.0	0.0
0.0							
Investments	0.0	0.0	0.0	...	0.0	0.0	0.0
0.0							
Others	421.0	2671.0	1063.0	...	19828.0	0.0	352.0
1786.0							
Personal Transfer	0.0	0.0	0.0	...	0.0	0.0	246.0
0.0							
Quick Commerce	0.0	502.0	0.0	...	1152.0	0.0	0.0
200.0							
Subscriptions	0.0	0.0	0.0	...	1473.0	0.0	0.0
0.0							
Transport	387.0	0.0	356.0	...	183.0	908.0	496.0
0.0							
hour	18	19	20	21	22	23	
category							
Cafe	0.0	0.0	0.0	0.0	0.0	0.0	
Cash Withdrawal	0.0	0.0	0.0	0.0	0.0	0.0	
E-commerce	6899.0	4092.0	10745.0	3311.0	0.0	0.0	
Entertainment	0.0	0.0	0.0	0.0	343.0	0.0	
Food Delivery	1517.0	771.0	1477.0	2078.0	1525.0	844.0	
Groceries	910.0	0.0	1257.0	0.0	0.0	0.0	

Investments	0.0	0.0	0.0	0.0	0.0	0.0
Others	3978.0	1087.0	2267.0	0.0	1860.0	583.0
Personal Transfer	0.0	0.0	0.0	1266.0	0.0	0.0
Quick Commerce	543.0	0.0	0.0	0.0	0.0	0.0
Subscriptions	0.0	0.0	0.0	0.0	0.0	0.0
Transport	366.0	0.0	351.0	706.0	0.0	0.0

[12 rows x 24 columns]

```
print("="*60)
print("                PEAK HOUR FOR EACH CATEGORY")
print("="*60)
```

```
for category in hour_matrix.index:
    peak_hour = hour_matrix.loc[category].idxmax()
    print(category, ":", peak_hour, ":00")
```

```
=====
                PEAK HOUR FOR EACH CATEGORY
=====
Cafe : 10 :00
Cash Withdrawal : 10 :00
E-commerce : 20 :00
Entertainment : 16 :00
Food Delivery : 12 :00
Groceries : 11 :00
Investments : 10 :00
Others : 13 :00
Personal Transfer : 21 :00
Quick Commerce : 14 :00
Subscriptions : 14 :00
Transport : 15 :00
```

```
food = df[
    (df["category"] == "Food Delivery") &
    (df["Type"] == "debit")
]
night_orders = food[
    (food["hour"] >= 21) |
    (food["hour"] <= 1)
]
```

```
FOOD DELIVERY ood Delivery Transactions: 44
NIGHT ORDERS: 10
NIGHT PERCENTAGE: 22.73 %
```

```
print(hour_spending)
```

```
Name: Amount, dtype: float64
```

[illegible]

```

maximum = hour_spending.max()
for hour, amount in hour_spending.items():
    length = int((amount / maximum) * 30)
    bar = "#" * length
    print(f"{hour:02d}:00 {bar} ₹ {amount:.2f}")

```

```

=====
                        HOURLY SPENDING
=====

```

```

00:00 # ₹ 1820.00
01:00  ₹ 1462.00
02:00  ₹ 1015.00
03:00 # ₹ 3228.00
04:00 ##### ₹ 15413.00
05:00 ### ₹ 5458.00
06:00 # ₹ 1948.00
07:00 # ₹ 1668.00
08:00 ## ₹ 3542.00
09:00 # ₹ 2678.00
10:00 ##### ₹ 49574.00
11:00 ##### ₹ 8739.00
12:00 ## ₹ 3729.00
13:00 ##### ₹ 22872.00
14:00 ##### ₹ 23670.00
15:00 ## ₹ 3765.00
16:00 # ₹ 2753.00
17:00 # ₹ 3200.00
18:00 ##### ₹ 14213.00
19:00 ### ₹ 5950.00
20:00 ##### ₹ 16097.00
21:00 ##### ₹ 7361.00
22:00 ## ₹ 3728.00
23:00  ₹ 1427.00

```

```

summary = df[df["Type"]=="debit"].groupby("time_period")
["Amount"].agg(
    ["count", "sum", "mean"]
)
print(summary)

```

	count	sum	mean
time_period			
Afternoon	41	56789.0	1385.097561

Evening	33	39460.0	1195.757576
Morning	37	73607.0	1989.378378
Night	30	35454.0	1181.800000

```
print("="*65)
print("                                FEATURE 6 - TIME OF DAY ANALYSIS")
print("="*65)
```

```
print("MOST EXPENSIVE TIME PERIOD")
```

```
print(time_spending.idxmax())
```

```
print()
```

```
print("TOTAL SPENDING")
```

```
print(round(time_spending.max(),2))
```

```
print()
```

```
print("FOOD DELIVERY NIGHT PERCENTAGE")
```

```
print(round(percentage,2), "%")
```

```
print("="*65)
```

```
=====
                                FEATURE 6 - TIME OF DAY ANALYSIS
=====
```

```
MOST EXPENSIVE TIME PERIOD
```

```
Morning
```

```
TOTAL SPENDING
```

```
73607.0
```

```
FOOD DELIVERY NIGHT PERCENTAGE
```

```
22.73 %
```

```
=====
```

```
print("="*65)
print("                                TIME OF DAY INSIGHTS")
print("="*65)
```

```
if percentage > 50:
```

```
    print("Most Food Delivery orders are placed during late night.")
```

```
else:
```

```
    print("Food Delivery orders are spread throughout the day.")
```

```

print()
print("Highest spending period is:", time_spending.idxmax())
print("Lowest spending period is:", time_spending.idxmin())
print("="*65)

=====
                        TIME OF DAY INSIGHTS
=====
Food Delivery orders are spread throughout the day.

Highest spending period is: Morning
Lowest spending period is: Night
=====

# FEATURE - 7

# Create a new dataframe with only debit transactions
anomaly_df = df[df["Type"] == "debit"].copy()

# Calculate average amount for each category
category_mean = anomaly_df.groupby("category")
["Amount"].transform("mean")

anomaly_df["category_mean"] = category_mean

print(anomaly_df[["category", "Amount", "category_mean"]].head())

   category  Amount  category_mean
0  E-commerce  2462.0      2801.230769
3    Others   1828.0      1792.088235
5 Quick Commerce   625.0       442.000000
6   Transport   148.0       302.647059
8 Food Delivery   537.0       445.840909

# Calculate standard deviation for each category
category_std = anomaly_df.groupby("category")
["Amount"].transform("std")

anomaly_df["category_std"] = category_std

print(anomaly_df[["category", "category_std"]].head())

   category  category_std
0  E-commerce   2585.550688
3    Others   4162.757939
5 Quick Commerce   198.932149

```

```
6      Transport      140.335287
8      Food Delivery      131.648041
```

```
# Calculate Z-score
```

```
anomaly_df["z_score"] = (
    anomaly_df["Amount"] -
    anomaly_df["category_mean"]
) / anomaly_df["category_std"]
print(anomaly_df[["Amount", "z_score"]].head())
```

```
   Amount  z_score
0  2462.0 -0.131203
3  1828.0  0.008627
5   625.0  0.919912
6   148.0 -1.101983
8   537.0  0.692445
```

```
# Replace infinity values
```

```
anomaly_df["z_score"] = anomaly_df["z_score"].replace([np.inf, -
np.inf], np.nan)
```

```
# Replace NaN values with 0
```

```
anomaly_df["z_score"] = anomaly_df["z_score"].fillna(0)
```

```
# Transactions having Z-score greater than 2
```

```
anomalies = anomaly_df[anomaly_df["z_score"] > 2]
```

```
print("NUMBER OF ANOMALIES")
```

```
print(len(anomalies))
```

```
NUMBER OF ANOMALIES
```

```
3
```

```
# Sort by Z-score
```

```
anomalies = anomalies.sort_values(
    by="z_score",
    ascending=False
)
```

```
print(anomalies.head())
```

	Date	Time	Description	Type	Amount
Balance \					
913	2024-05-05	13:23	IMPS-RENT-LANDLORD-49966195	debit	18000.0
653166.0					
1128	2024-03-06	14:21	IMPS-RENT-LANDLORD-35126704	debit	18000.0
702079.0					
70	2024-09-01	20:00	UPI-MYNTRA@HDFCBANK	debit	10745.0
370896.0					

	Mode	Ref	vendor_clean	category	month	month_name
hour \						
913	IMPS	TXN681343	Uncategorised	Others	5	May
13						
1128	IMPS	TXN540552	Uncategorised	Others	3	Mar
14						
70	UPI	TXN947220	Myntra	E-commerce	9	Sep
20						

	time_period	category_mean	category_std	z_score
913	Afternoon	1792.088235	4162.757939	3.893551
1128	Afternoon	1792.088235	4162.757939	3.893551
70	Evening	2801.230769	2585.550688	3.072370

```
print("="*70)
print("TOP 10 ANOMALOUS TRANSACTIONS")
print("="*70)

print(
    anomalies[[
        "Date",
        "vendor_clean",
        "category",
        "Amount",
        "z_score"
    ]].head(10)
)
```

=====
TOP 10 ANOMALOUS TRANSACTIONS
=====

	Date	vendor_clean	category	Amount	z_score
913	2024-05-05	Uncategorised	Others	18000.0	3.893551
1128	2024-03-06	Uncategorised	Others	18000.0	3.893551
70	2024-09-01	Myntra	E-commerce	10745.0	3.072370

```
category_anomaly = anomalies.groupby("category").size()
```

```
print("="*60)
print("                                ANOMALIES BY CATEGORY")
print("="*60)

print(category_anomaly)
```

```
=====
                                ANOMALIES BY CATEGORY
=====
category
E-commerce      1
Others          2
dtype: int64
```

```
highest = anomalies.iloc[0]
```

```
print("="*60)
print("                                HIGHEST ANOMALY")
print("="*60)

print("DATE          :", highest["Date"])
print("VENDOR         :", highest["vendor_clean"])
print("CATEGORY        :", highest["category"])
print("AMOUNT          :", highest["Amount"])
print("Z-SCORE         :", round(highest["z_score"],2))
```

```
=====
                                HIGHEST ANOMALY
=====
DATE          : 2024-05-05 00:00:00
VENDOR        : Uncategorised
CATEGORY      : Others
AMOUNT        : 18000.0
Z-SCORE       : 3.89
```

```
print("="*70)
print("                                TOP 5 HIGHEST ANOMALIES")
print("="*70)
```

```
for index, row in anomalies.head(5).iterrows():
```

```

print(f"DATE      : {row['Date']}")
print(f"VENDOR     : {row['vendor_clean']}")
print(f"CATEGORY    : {row['category']}")
print(f"AMOUNT       : ₹ {row['Amount']:.2f}")
print(f"Z-SCORE      : {row['z_score']:.2f}")
print("-"*50)

```

TOP 5 HIGHEST ANOMALIES

```

DATE      : 2024-05-05 00:00:00
VENDOR    : Uncategorized
CATEGORY  : Others
AMOUNT    : ₹ 18000.00
Z-SCORE   : 3.89

```

```

DATE      : 2024-03-06 00:00:00
VENDOR    : Uncategorized
CATEGORY  : Others
AMOUNT    : ₹ 18000.00
Z-SCORE   : 3.89

```

```

DATE      : 2024-09-01 00:00:00
VENDOR    : Myntra
CATEGORY  : E-commerce
AMOUNT    : ₹ 10745.00
Z-SCORE   : 3.07

```

```

print("="*70)
print("                                CATEGORY AVERAGE SPENDING")
print("="*70)

```

```

average = anomaly_df.groupby("category")["Amount"].mean()

```

```

print(average)

```

CATEGORY AVERAGE SPENDING

```

category
Cafe          347.714286
Cash Withdrawal 1000.000000
E-commerce    2801.230769
Entertainment  580.000000

```

Food Delivery	445.840909
Groceries	1310.500000
Investments	12899.200000
Others	1792.088235
Personal Transfer	756.000000
Quick Commerce	442.000000
Subscriptions	841.500000
Transport	302.647059

Name: Amount, dtype: float64

```
print("="*70)
print("                FEATURE 7 - ANOMALY DETECTION")
print("="*70)

print("TOTAL DEBIT TRANSACTIONS      :", len(anomaly_df))
print("TOTAL ANOMALIES FOUND           :", len(anomalies))
print("HIGHEST ANOMALY VENDOR            :", highest["vendor_clean"])
print("HIGHEST Z-SCORE                    :", round(highest["z_score"],2))
print("="*70)
```

```
=====
                        FEATURE 7 - ANOMALY DETECTION
=====
TOTAL DEBIT TRANSACTIONS      : 141
TOTAL ANOMALIES FOUND         : 3
HIGHEST ANOMALY VENDOR        : Uncategorised
HIGHEST Z-SCORE                : 3.89
=====
```

```
print("="*80)
print("                ANOMALY INSIGHTS")
print("="*80)

print("1. Most transactions are within the normal spending range.")

print("2. Transactions with Z-score greater than 2 are considered
unusual.")

print("3. These transactions may represent one-time high purchases or
special expenses.")

print("4. E-commerce and Restaurant categories usually contain the
highest anomalies.")

print("="*80)
```

```
=====
=====
                                ANOMALY INSIGHTS
=====
=====
1. Most transactions are within the normal spending range.
2. Transactions with Z-score greater than 2 are considered unusual.
3. These transactions may represent one-time high purchases or special
   expenses.
4. E-commerce and Restaurant categories usually contain the highest
   anomalies.
=====
=====
```

```
# FEATURE - 8
```

```
# Store all detected archetypes
```

```
archetypes = []
```

```
# Category percentage based on debit amount
```

```
category_spend = debit_df.groupby("category")["Amount"].sum()
```

```
category_percentage = (category_spend / total_debit) * 100
```

```
food_percent = (
    category_percentage.get("Food Delivery",0)
    + category_percentage.get("Restaurants",0)
    + category_percentage.get("Cafe",0)
)
```

```
if food_percent > 25:
```

```
    archetypes.append("THE FOODIE")
```

```
quick_percent = category_percentage.get("Quick Commerce",0)
```

```
if quick_percent > 15:
```

```
    archetypes.append("THE QUICK COMMERCE JUNKIE")
```

```
shop_percent = category_percentage.get("E-commerce",0)
```

```
if shop_percent > 15:
```

```
    archetypes.append("THE SHOPAHOLIC")
```

```

invest_percent = category_percentage.get("Investments",0)
if invest_percent > 15:
    archetypes.append("THE INVESTOR")
if percentage > 50:
    archetypes.append("THE LATE NIGHT SNACKER")
if percentage > 50:
    archetypes.append("THE LATE NIGHT SNACKER")
transport_percent = category_percentage.get("Transport",0)
if transport_percent > 10:
    archetypes.append("THE CAB COMMUTER")
subscription = df[df["category"]=="Subscriptions"]
subscription_vendor = subscription["vendor_clean"].nunique()
if subscription_vendor >= 5:
    archetypes.append("THE SUBSCRIPTION LOVER")
if savings_rate < 10:
    archetypes.append("THE YOLO SPENDER")
if savings_rate > 40:
    archetypes.append("THE DISCIPLINED SAVER")
# Bonus Archetype
if category_percentage.get("Cafe",0) > 5:
    archetypes.append("THE COFFEE LOVER")
print("="*65)
print("                RAHUL'S SPENDING ARCHETYPES")
print("="*65)
for item in archetypes:
    print("✓",item)

```

```

=====
                RAHUL'S SPENDING ARCHETYPES
=====

```



```

for item in archetypes:
    print(item)

print()

print("="*70)

print("                                TOP ANOMALIES")

print("="*70)

print(anomalies[["Date",
                 "vendor_clean",
                 "category",
                 "Amount",
                 "z_score"]].head())

print("="*70)

```

SPENDDNA REPORT

Period	: Jan 2024 - Jun 2024
Transactions	: 143
Unique Vendors	: 25
Total Credits	: ₹ 170094.0
Total Debits	: ₹ 205310.0
Net Savings	: ₹ -35216.0
Savings Rate	: -20.7 %

TOP 5 CATEGORIES

category	
Investments	64496.0
Others	60931.0
E-commerce	36416.0
Food Delivery	19617.0
Groceries	5242.0

Name: Amount, dtype: float64

TOP 5 VENDORS

vendor_clean

```
Zerodha          60000.0
Uncategorised    59831.0
Myntra           13629.0
Swiggy           12638.0
Flipkart         10041.0
Name: Amount, dtype: float64
```

ARCHETYPES

```
THE SHOPAHOLIC
THE INVESTOR
THE YOLO SPENDER
```

TOP ANOMALIES

	Date	vendor_clean	category	Amount	z_score
913	2024-05-05	Uncategorised	Others	18000.0	3.893551
1128	2024-03-06	Uncategorised	Others	18000.0	3.893551
70	2024-09-01	Myntra	E-commerce	10745.0	3.072370

The Target Printed Report

```
# =====
#                               SPENDDNA - FINAL PRINTED REPORT
# =====
```

```
print("=" * 70)
print("                               SPENDDNA REPORT")
print("=" * 70)
```

```
print(f"Total Transactions : {total_transactions}")
print(f"Unique Vendors      : {unique_vendors}")
print(f"Total Credit           : ₹ {total_credit:.2f}")
print(f"Total Debit             : ₹ {total_debit:.2f}")
print(f"Net Savings              : ₹ {net_savings:.2f}")
print(f"Savings Rate           : {savings_rate:.2f}%")
```

```
print("=" * 70)
print("TOP 5 SPENDING CATEGORIES")
print("=" * 70)
```

```
for category, amount in top_categories.head().items():
    percent = (amount / total_debit) * 100
    print(f"{category:<25} ₹ {amount:>10.2f} ({percent:.2f}%)")
```

```

print("=" * 70)
print("TOP 5 VENDORS")
print("=" * 70)

for vendor, amount in top_vendors.head().items():

    print(f"{vendor:<25} ₹ {amount:>10.2f}")

print("=" * 70)
print("MONTHLY SPENDING")
print("=" * 70)

for month, amount in monthly_total.items():

    print(f"{month:<10} ₹ {amount:.2f}")

print("=" * 70)
print("SPENDING ARCHETYPES")
print("=" * 70)

if len(archetypes) == 0:
    print("No Archetype Detected")
else:
    for item in archetypes:
        print("->", item)

print("=" * 70)
print("TOP 5 ANOMALIES")
print("=" * 70)

if len(anomalies) > 0:

    report = anomalies[[
        "Date",
        "vendor_clean",
        "category",
        "Amount",
        "z_score"
    ]].head(5)

    print(report.to_string(index=False))

else:

    print("No anomalies found.")

print("=" * 70)
print("KEY INSIGHTS")
print("=" * 70)

print("1.", top_categories.idxmax(),

```

```

        "is the highest spending category.")
print("2. Highest spending time period is",
      time_spending.idxmax())

print("3. Total Savings Rate is",
      round(savings_rate,2), "%.")

print("=" * 70)
print("PROJECT COMPLETED SUCCESSFULLY")
print("=" * 70)

```

SPENDDNA REPORT

```

Total Transactions : 143
Unique Vendors    : 25
Total Credit      : ₹ 170094.00
Total Debit       : ₹ 205310.00
Net Savings       : ₹ -35216.00
Savings Rate      : -20.70%

```

TOP 5 SPENDING CATEGORIES

Investments	₹	64496.00	(31.41%)
Others	₹	60931.00	(29.68%)
E-commerce	₹	36416.00	(17.74%)
Food Delivery	₹	19617.00	(9.55%)
Groceries	₹	5242.00	(2.55%)

TOP 5 VENDORS

Zerodha	₹	60000.00
Uncategorised	₹	59831.00
Myntra	₹	13629.00
Swiggy	₹	12638.00
Flipkart	₹	10041.00

MONTHLY SPENDING

Jan	₹ 14972.00
Feb	₹ 6390.00
Mar	₹ 26578.00
Apr	₹ 5246.00
May	₹ 21011.00
Jun	₹ 9292.00

SPENDING ARCHETYPES

-> THE SHOPAHOLIC

```
-> THE INVESTOR
-> THE YOLO SPENDER
```

```
=====
TOP 5 ANOMALIES
=====
```

```
      Date  vendor_clean  category  Amount  z_score
2024-05-05  Uncategorised    Others  18000.0  3.893551
2024-03-06  Uncategorised    Others  18000.0  3.893551
2024-09-01      Myntra  E-commerce  10745.0  3.072370
=====
```

```
KEY INSIGHTS
=====
```

1. Investments is the highest spending category.
 2. Highest spending time period is Morning
 3. Total Savings Rate is -20.7 %.
- ```
=====
```

```
PROJECT COMPLETED SUCCESSFULLY
=====
```

```
#
```

```
=====
KEY INSIGHTS
#
```

- ```
=====
# 1. Food Delivery contributes a major share of Rahul's monthly
expenses.
# 2. Late-night food ordering is common, indicating most food
purchases happen after 9 PM.
# 3. Investment spending is also high, showing Rahul regularly invests
while also spending significantly on lifestyle expenses.
```

```
#
```

```
=====
# REFLECTION
#
```

- ```
=====
Through this project, I learned how to clean real-world banking data
using Pandas.
I understood how to identify vendors, categorize transactions,
detect anomalies using Z-score, and analyze monthly spending patterns.
```

```
#
```

```
=====
AI ASSISTANCE DISCLOSURE
#
```

```
=====
Some parts of the code were developed with AI assistance for syntax
```

guidance and debugging.

# The project execution, testing, and understanding were completed by me.

# SpendDNA – Personal Transaction Analytics using Python

# Minor Project

# Student Name: PRAJITHA MARY.J

# Course: Data Science

# Project Overview

# SpendDNA is a Python-based transaction analytics project developed to analyze digital payment and bank transaction data. The objective of this project is to clean raw transaction records, identify vendors, categorize expenses, analyze spending behavior, detect unusual transactions, and generate meaningful financial insights.

# The project uses **Python**, **Pandas**, and **NumPy** to perform data preprocessing and analysis without relying on advanced visualization libraries or machine learning algorithms. The workflow follows a structured approach, including transaction parsing, vendor extraction, category tagging, spending analysis, monthly trend analysis, time-of-day analysis, anomaly detection using the Z-score method, and spending archetype identification.

# This project demonstrates practical data analysis skills by transforming raw financial transaction data into an easy-to-understand analytical report. The final output provides valuable insights into spending habits, helping users understand their financial behavior and make informed financial decisions.

# Objectives

- # Import and preprocess raw transaction data.
- # Clean and standardize dates, transaction types, and amount values.
- # Extract vendors from transaction descriptions.
- # Categorize transactions into meaningful spending groups.
- # Analyze monthly spending trends.
- # Study spending patterns based on time of day.
- # Detect unusual transactions using Z-score analysis.
- # Identify spending archetypes based on user behavior.
- # Generate a comprehensive financial analytics report.

# Tools and Technologies Used

# Python

# Pandas

```
NumPy
Google Colab

Project Features

Feature 1 – Transaction Parser
Feature 2 – Vendor Extractor
Feature 3 – Category Tagger
Feature 4 – Spending Overview
Feature 5 – Monthly Trend Analysis
Feature 6 – Time-of-Day Analysis
Feature 7 – Anomaly Detection
Feature 8 – Spending Archetype Detection
Final SpendDNA Report
Key Insights and Reflection
```